

MASTERCLASS - 12

PROMPT KIT

AI CODING AGENTS

By MissAlpha™ AI Labs

Learn | Build | Grow

MISSALPHA™ QUICKCART

AI Coding Agents Masterclass —

One project → one workflow → one tool for each responsibility.

Project

AI-Powered Quick-Commerce E-Commerce Website

React

+

Python FastAPI

+

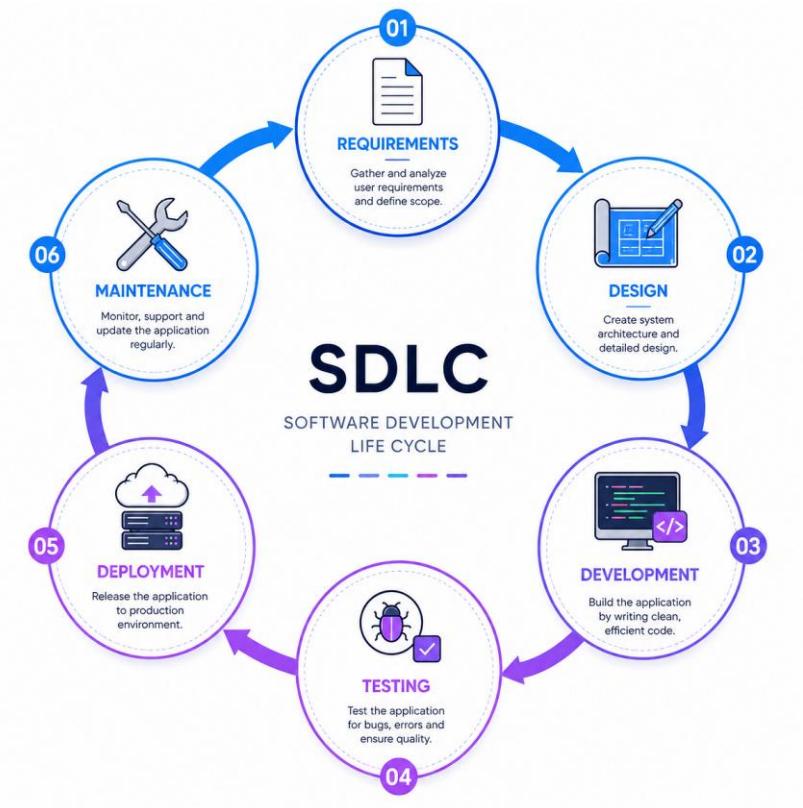
PostgreSQL

+

LLM API

+

AI Coding Agent



The project grow like this:

IDEA

↓

REQUIREMENTS

↓

ARCHITECTURE

↓

PLAN

↓

REACT FRONTEND

↓

PYTHON BACKEND

↓

DATABASE

↓

API INTEGRATION

↓

AI FEATURE

↓

DEBUG

↓

TEST

↓

REVIEW

↓

SECURE

↓

DEPLOY

---

1. WHAT EACH AI TOOL DOES

Tool	What it does	Simple explanation
ChatGPT	Planning, explanation, requirements	Think with me
Claude	Architecture, reasoning, review	Analyze with me
Cursor Agent	Build, edit, run, debug	Build with me
Claude Code	Terminal-based coding agent	Work on my codebase
OpenAI Codex	Software engineering tasks	Complete coding tasks
GitHub Copilot	Coding assistance + GitHub workflows	Assist while coding
Postman	API testing	Test my APIs
PostgreSQL	Database	Store my data
Git/GitHub	Version control	Save and track my code
LLM API	AI inside application	Make my app intelligent

ChatGPT → Cursor Agent → Postman → GitHub → LLM API

Keep Claude Code and Codex as alternatives rather than switching between five coding agents during the live build.

Your source plan similarly identifies Cursor Agent as the primary agent and Claude Code/Codex as alternatives.

WHO DOES WHAT?

CHATGPT / CLAUDE

↓

THINK + PLAN + EXPLAIN

↓

CURSOR AGENT

↓

BUILD + EDIT + RUN + DEBUG

↓

POSTMAN

↓

TEST APIs

↓

GITHUB

↓

VERSION CONTROL

↓

LLM API

↓

ADD AI TO THE APPLICATION

↓

HUMAN

↓

REVIEW + APPROVE + DECIDE

Now Let's START THE QUICKCART PROJECT

TASK 1 — THINK BEFORE YOU CODE

Goal

Understand the business problem.

AI TOOL

ChatGPT

Why?

Because at this stage we don't need an AI coding agent.

We need thinking and planning.

STEP 1.1 — IDEA

Idea

Build a modern grocery quick-commerce platform with an AI shopping assistant.

STEP 1.2 — REQUIREMENTS

Tool: ChatGPT

Prompt:

Act as a Senior Product Manager.

We are building an AI-powered quick-commerce

platform called QuickCart.

Users should be able to:

- Browse grocery products

- Search products

- Filter categories

- View product details

- Add products to cart

- Update quantity

- Remove products

- Proceed toward checkout

- Use an AI shopping assistant

Target users:

- College students

- Working professionals

- Families

- Everyday grocery shoppers

Create a simple Product Requirements Document.

Include:

1. Problem

2. Target users

3. User personas

4. Features

5. User stories

6. Functional requirements

7. Non-functional requirements

8. MVP

9. Future features

10. Acceptance criteria

Do NOT write code.

We learnt here :

IDEA

↓

REQUIREMENTS

STEP 1.3 — ARCHITECTURE

Tool: ChatGPT / Claude

Prompt:

Act as a Senior Full-Stack Architect.

Using the approved QuickCart requirements,

design the application architecture.

Frontend:

React + Tailwind CSS

Backend:

Python + FastAPI

Database:

PostgreSQL

AI:

LLM API

Development:

Git + GitHub

API Testing:

Postman

Show:

1. System architecture

2. Frontend architecture

3. Backend architecture

4. Database entities

5. API structure

6. AI architecture

7. Authentication

8. Security

9. Folder structure

Explain everything in beginner-friendly language.

Do not write implementation code.

STEP 1.4 — CREATE THE DEVELOPMENT PLAN

Tool: Cursor Agent — Plan Mode

Now we move from thinking to agentic development.

Cursor's planning capability is useful here because the agent can plan complex multi-file work before implementation.

Prompt

You are the AI Coding Agent for QuickCart.

Do NOT start coding yet.

First create an implementation plan.

Break the project into:

1. Project setup

2. React storefront

3. Product catalog

4. Search

5. Filtering

6. Cart

7. FastAPI backend

8. PostgreSQL database

9. Frontend-backend integration

10. AI Shopping Assistant

11. Testing

12. Security

13. Deployment

For each step provide:

- Goal

- Files affected

- Dependencies

- Implementation steps

- Validation

- Possible risks

Wait for approval before implementation.

"The agent is not coding yet. It is planning."

TASK 2 — BUILD THE FRONTEND

STEP 2.1 — CREATE REACT APPLICATION

Tool

Cursor Agent

Technologies

- **React**
- **Tailwind CSS**

Prompt:

Act as a Senior React Developer.

Implement the QuickCart frontend.

Before coding:

1. Inspect the project.

2. Review the existing structure.

3. Identify files to create/change.

4. Create an implementation plan.

Then build:

- Header

- Search bar

- Location selector

- Categories

- Promotional banner

- Product cards

- Product grid

- Cart indicator

- Footer

Use reusable React components.

Use original QuickCart branding.

Requirements:

- Responsive

- Accessible

- Clean component structure

- Loading states

- Empty states

After implementation:

1. Run the application.

2. Check for errors.

3. Fix errors.

4. Summarize changes.

Now see:

EMPTY FOLDER

↓

REACT

↓

QUICKCART UI

STEP 2.2 — ADD INTERACTION

Tool

Cursor Agent

Don't create a new prompt from scratch.

Continue the same agent session.

Now implement the next frontend milestone.

Add:

1. Product search

2. Category filtering

3. Price sorting

4. Discount badges

5. Quantity controls

6. Add to cart

7. Cart count

8. Empty cart

9. Loading state

10. Mobile navigation

Important:

- Inspect existing code first.

- Reuse components.

- Do not rewrite working functionality.

- Make incremental changes.

- Run the application.

- Fix errors.

One prompt = one meaningful development increment.

TASK 3 — BUILD THE BACKEND

"Frontend is what users see. Backend is where application logic lives."

STEP 3.1 — FASTAPI

Tool

Cursor Agent

Technology

Python + FastAPI

Prompt:

Act as a Senior Python Backend Engineer.

Build the QuickCart backend using:

Python + FastAPI.

Create modules for:

- Products

- Categories

- Cart

- Users

- Orders

Create APIs:

GET /api/products

GET /api/products/{id}

GET /api/categories

POST /api/cart/items

GET /api/cart

PATCH /api/cart/items/{id}

DELETE /api/cart/items/{id}

Use:

- Pydantic

- Routes

- Schemas

- Services

- Database layer

- Validation

- Error handling

- Proper HTTP status codes

- CORS

- Environment variables

- API documentationFirst inspect the project.

Implement incrementally.

Then:

- Start the server.

- Test APIs.

- Fix errors.

STEP 3.2 — DATABASE

Tool

Cursor Agent

Technology

PostgreSQL

Prompt:

Implement the QuickCart PostgreSQL database.

Entities:

User

Category

Product

Cart

CartItem

Order

OrderItem

Create:

- Primary keys

- Foreign keys

- Relationships

- Validation

- Useful indexes

Create realistic grocery seed data.

Verify:

1. Database connection

2. Database setup

3. CRUD operations

4. Backend integration

Never store passwords or API keys in plain text.

The source project plan uses these same core entities and database responsibilities.

STEP 3.3 — CONNECT FRONTEND + BACKEND

Tool

Cursor Agent

Prompt:

Act as a Senior Full-Stack Engineer.

Connect the QuickCart React frontend

to the FastAPI backend.

Replace static product data with API data.

Implement:

1. Fetch products

2. Fetch categories

3. Search

4. Add to cart

5. Retrieve cart

6. Update quantity

7. Remove item

8. Loading state

9. Error state

10. Empty state

Requirements

- Create reusable API service functions.

- Keep API logic separate from UI.

- Use environment variables.

- Handle network errors.

- Do not hardcode localhost throughout components.

Run frontend and backend.

Verify the complete flow.

Now see:

REACT

↓

HTTP REQUEST

↓

FASTAPI

↓

BUSINESS LOGIC

↓

POSTGRESQL

↓

JSON RESPONSE

↓

REACT

"Now our UI is connected to real software infrastructure."

STEP 3.4 — TEST THE APIs

Tool

Postman

This is important:

Don't use AI for everything.

See that professional developers use specialized tools.

Test:

GET /products

GET /categories

POST /cart/items

GET /cart

PATCH /cart/items

DELETE /cart/items

Cursor builds the API. Postman verifies the API.

TASK 4 — ADD AI INTELLIGENCE

Now we make QuickCart genuinely AI-powered.

Tool Map

Job	Tool
Design AI feature	ChatGPT / Claude
Implement AI backend	Cursor Agent
Build AI UI	Cursor Agent
AI reasoning	LLM API
Product data	PostgreSQL
Validation	FastAPI
Testing	Postman + automated tests

STEP 4.1 — DESIGN AI FEATURE

Tool

ChatGPT / Claude

Ask:

Design an AI Shopping Assistant for QuickCart.

Users can ask:

"Breakfast for 3 people under ₹500."

"Vegetarian dinner for 4 people."

"Healthy snacks under ₹300."

Design:

1. User experience

2. AI workflow

3. Backend architecture

4. API endpoint

5. Input schema

6. Output schema

7. Product matching

8. Budget handling

9. Validation

10. Failure handling

Important:

AI must not invent:

- Products

- Prices

- Product IDs

- Stock

Products must come from our database.

Return structured JSON.

Do not implement yet.

STEP 4.2 — BUILD AI BACKEND

Tool

Cursor Agent

Implement:

POST /api/ai/shopping-assistant

Input:

```
{
  "query": "breakfast for 3 people under 500",
  "preferences": {
    "diet": "vegetarian"
  }
}
```

Workflow:

1. Validate request.

2. Retrieve relevant products from PostgreSQL.

3. Send relevant product information to the LLM.

4. Ask the LLM to recommend only supplied products.

5. Require structured JSON.

6. Validate AI response.

7. Reject unknown products.

8. Calculate totals using database prices.

9. Return recommendations.

10. Handle AI/API failures.

The database is the source of truth.

Never trust AI for:

- Product price

- Product ID

- Stock

- Final total

Add tests.

"AI recommends. Our application validates."

STEP 4.3 — BUILD AI FRONTEND

Tool

Cursor Agent

Prompt:

Create an AI Shopping Assistant

inside the QuickCart React application.

Add:

- Chat interface

- User message

- AI response

- Suggested prompts

- Recommended products

- Product price

- Quantity

- Add to cart

- Loading state

- Error state

Suggested prompts:

"Breakfast for 3 under ₹500"

"Vegetarian dinner for 4"

"Healthy snacks under ₹300"

Use actual product information

returned by our backend.

Allow recommended products

to be added to the cart.

Do not redesign unrelated features.

THE WOW MOMENT

"I need a vegetarian dinner for 4 people under ₹500."

watch:

USER

↓

NATURAL LANGUAGE

↓

AI

↓

PRODUCT DATABASE

↓

MATCHING

↓

RECOMMENDATION

↓

VALIDATION

↓

PRODUCT CARDS

↓

ADD TO CART

"Now We have moved from AI-generated code to an AI-powered application."

TASK 5 — DEBUG WITH AN AI CODING AGENT

Tool

Cursor Agent

Introduce a controlled bug.

Example:

Wrong API endpoint

↓

404 Error

Then:

Act as a Senior Debugging Engineer.

Here is the error

[PASTE ERROR]

Affected feature:

[FEATURE]

Expected:

[EXPECTED]

Actual:

[ACTUAL]

Do not immediately rewrite the code.

First:

1. Find root cause.

2. Explain the problem.

3. Identify affected files.

4. Propose smallest safe fix.

5. Implement the fix.

6. Run the relevant test.

7. Verify the original functionality.

Do not modify unrelated code.

Student learns:

ERROR

↓

ROOT CAUSE

↓

FIX

↓

TEST

↓

VERIFY

STEP 5.2 — TEST THE APPLICATION

AI Tool

Cursor Agent

Testing tools

Postman
Pytest / frontend test framework

Prompt:

Act as a Senior QA Engineer.

Create a QuickCart testing strategy.

Test:

PRODUCTS

- Loading

- Search

- Filtering

- Invalid product

CART

- Add

- Increase quantity

- Decrease quantity

- Remove

- Empty cart

- Invalid quantity

API

- Invalid request

- Missing fields

- Server error

AI

- Valid query

- Empty query

- Budget limit

- No matching products

- Very long query

- Invalid AI response

- AI service unavailable

For each test provide**:**

Test ID

Scenario

Input

Expected result

Priority

Then implement the highest-priority tests.

"Code generated ≠ Code verified."

STEP 5.3 — CODE REVIEW

Tool

Cursor Agent / Claude Code

Act as a Senior Software Engineer.

Review the entire QuickCart project.

Inspect:

- React

- FastAPI

- Database

- APIs

- AI integration

- Authentication

- Validation

- Error handling

- Environment variables

Look for:

- Bugs

- Security issues

- Hardcoded secrets

- Duplicate code

- Poor API design

- Missing validation

- AI hallucination risks

- Performance problems

Categorize:

CRITICAL

HIGH

MEDIUM

LOW

Do not modify code.

Give:

Location

Problem

Impact

Recommended fix

"AI can review code, but the developer decides whether the change should be accepted."

STEP 5.4 — SECURITY

Tool

Cursor Agent / Claude Code

Perform a security review of QuickCart.

Check:

- API keys

- Git secrets

- Environment variables

- SQL injection

- XSS

- CORS

- Authentication

- Authorization

- AI permissions

- Prompt injection

- Untrusted AI output

- Data leakage

- Dependency vulnerabilities

For every issue provide:

Risk

Evidence

Impact

Mitigation

Never print secret values.

Do not make destructive changes.

Create a security remediation plan.

Your existing masterclass framework emphasizes protecting Code + Data + Tools + Secrets, with a human checkpoint before execution.

STEP 5.5 — DEPLOYMENT

Tool

Cursor Agent

Supporting tools

Git → GitHub → CI/CD → Hosting

Prompt:

Act as a Senior DevOps Engineer.

Prepare QuickCart for production deployment.

Review:

- React frontend

- FastAPI backend

- PostgreSQL

- Environment variables

- API configuration

- CORS

- Build process

- Logging

- Error handling

Create:

1. Production environment template

2. Frontend build configuration

3. Backend deployment configuration

4. Database checklist

5. Production CORS configuration

6. Health-check endpoint

7. Deployment checklist

8. Git workflow

9. CI/CD recommendation

10. Rollback plan

Do not deploy automatically.

Identify everything that requires human approval.

Then:

CODE

↓

GIT

↓

GITHUB

↓

CI/CD

↓

DEPLOY

↓

USERS

COMPLETE ROADMAP

BUILD QUICKCART WITH AI

#	Step	AI Agent / Tool	Technology	Student Outcome
01	💡 Idea	ChatGPT	—	Understand problem
02	📋 Requirements	ChatGPT	PRD	Define product
03	🏗️ Architecture	ChatGPT / Claude	System design	Design solution
04	🧮 Planning	Cursor Agent	Plan Mode	Break project
05	🎨 Frontend	Cursor Agent	React	Build UI
06	✨ UI Features	Cursor Agent	React	Search + Cart
07	⚙️ Backend	Cursor Agent	FastAPI	Build APIs
08	🗄️ Database	Cursor Agent	PostgreSQL	Store data
09	🔌 Integration	Cursor Agent	REST API	Connect systems

10	🧪 API Testing	Postman	REST	Verify APIs
11	🤖 AI Design	ChatGPT / Claude	AI architecture	Design AI
12	🧠 AI Backend	Cursor Agent	LLM API	AI logic
13	💬 AI UI	Cursor Agent	React	AI assistant
14	🐛 Debug	Cursor Agent	DevTools	Fix bugs
15	🧪 Testing	Cursor Agent	Pytest/tests	Verify
16	🔍 Review	Cursor / Claude Code	GitHub	Code quality
17	🔐 Security	Cursor / Claude Code	Security	Reduce risks
18	🚀 Deploy	Cursor Agent	GitHub/CI/CD	Production-ready

REMEMBER ONLY 5 AI ROLES

1. CHATGPT / CLAUDE

Think + Plan

2. CURSOR AGENT

Build + Modify + Run + Debug

3. POSTMAN

Test APIs

4. GITHUB

Track + Collaborate

5. LLM API

Make the application intelligent

And above everything:

HUMAN DEVELOPER

Decide + Review + Validate + Approve

FINAL MASTERCLASS FLOW

QUICKCART



THINK WITH AI

ChatGPT / Claude

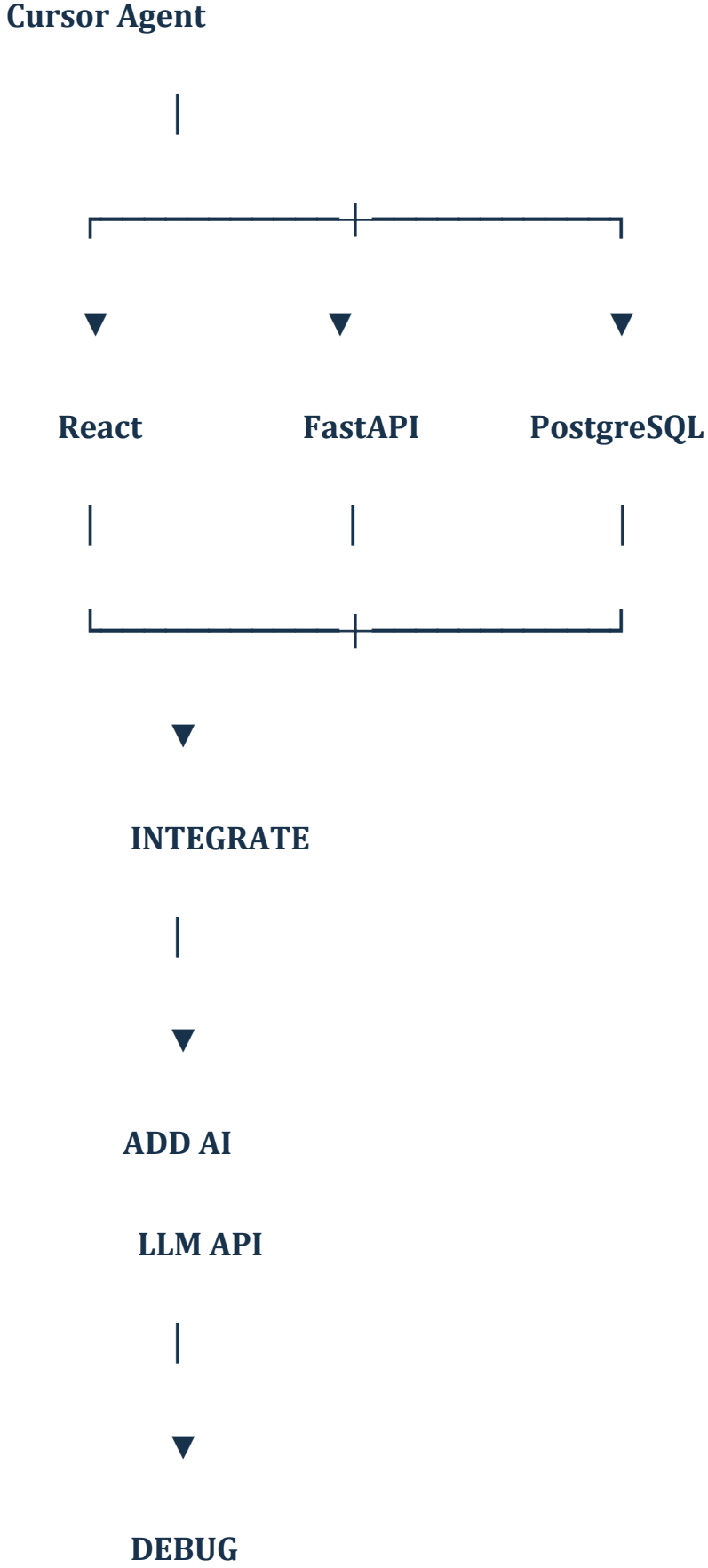


PLAN WITH AI

Cursor Plan Mode



BUILD WITH AI


Cursor Agent
React
FastAPI
PostgreSQL
INTEGRATE
ADD AI
LLM API
DEBUG

Cursor Agent



TEST

Postman + Tests



REVIEW

Cursor / Claude Code



SECURE



SHIP

GitHub + CI/CD

AI USER → AI-ASSISTED DEVELOPER → AI CODING AGENT USER → AI BUILDER

The practical message is not "AI can create an e-commerce website. I can use different AI agents and development tools at different stages of the SDLC to build a real software product."

MissAlpha

MissAlpha™ AI Labs

Learn | Build | Grow

Thank You!

For being a part of this learning journey!

- Learn New Skills
- Apply in Real Scenarios
- Enhance Productivity
- Grow Your Career



WhatsApp Channel | LinkedIn Profile | GitHub | YouTube | Instagram